Milestone: 20GB Reached

We’ve reached 20GB of stored data — thank you for helping us grow!

Patreon Ko-fi Liberapay
Close
Freedium

- ko-fi
- librepay
- patreon

< Go to the original

Need	Best fit	Why
Long-running agent conversations with lots of tools	Observational Memory	Compresses raw history + tool exhaust; stable + cacheable context
Small structured state (names, preferences, goals)	Working memory	Designed for small, structured data; not a growing event log
Retrieve specific past turns based on similarity	Semantic recall (RAG)	Strong recall, but dynamic injection + cache invalidation cost
You're summarizing constantly to survive context limits	Observational Memory	OM manages history automatically via thresholds

Observational Memory for Long-Running Agents — "This wasn't the plan!"

Always-on agents have unbounded context growth problem.

Agent Native Follow
~9 min read · February 16, 2026 (Updated: February 17, 2026) · Free: No

Tool calls can easily dump 50k tokens into your thread, and **helpful traces** and **file scans** metastasize.

And eventually your agent flips into **compaction / summary mode** and starts making decisions based on… what, exactly?

I've been burned by this enough times that I now treat memory as a first-class system.

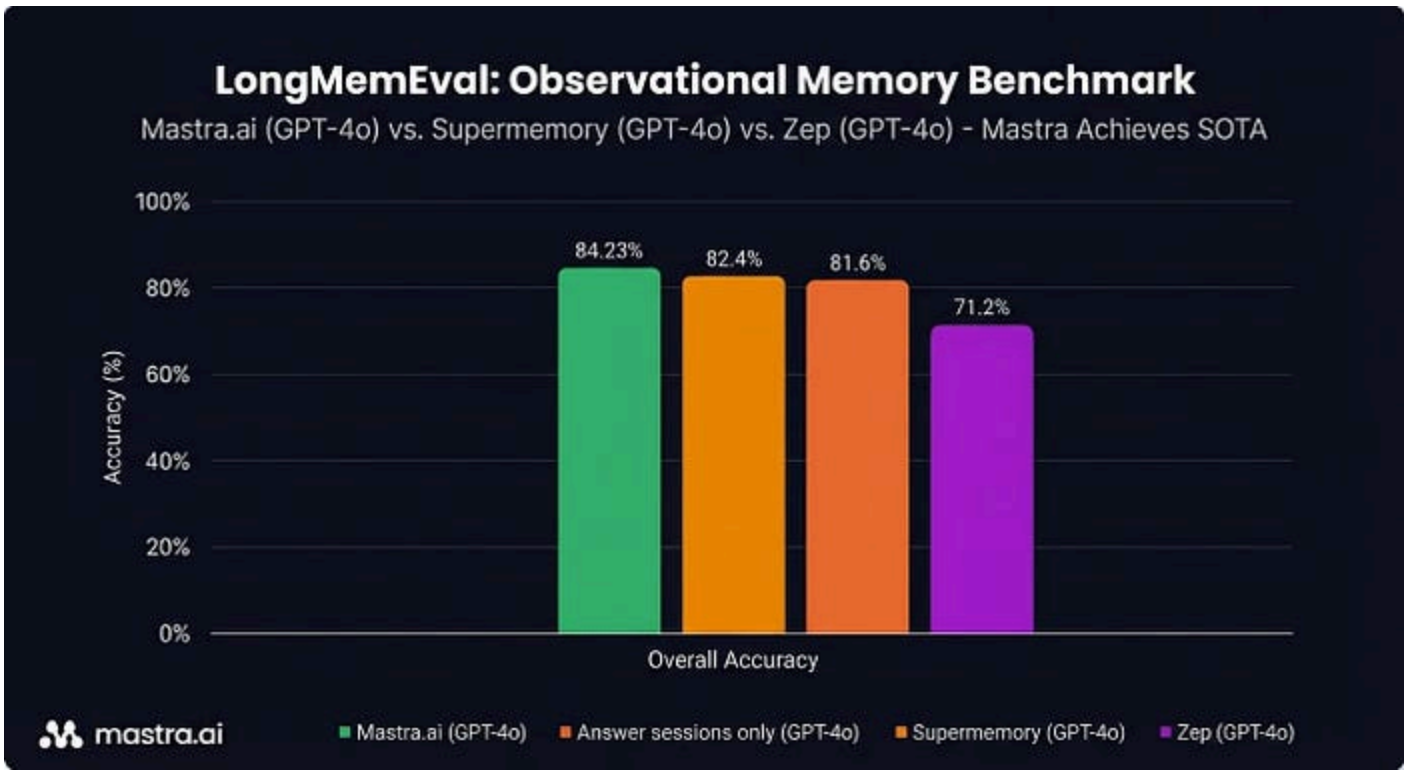
Observational Memory (OM) is Mastra's approach to long-context agent memory.

Instead of dragging raw message history forever, OM maintains an **observation log** built by two background workers:

- **Observer**: watches the conversation + tool outputs and writes structured **observations**
- **Reflector**: merges related observations, extracts patterns, and condenses aggressively

Context window stays **stable and prompt-cacheable** as your agent runs for hours/days.

In **LongMemEval**, OM even beats a gpt-4o "oracle" setup (the one that gets only the answer sessions) by ~2 points.



> Surprisingly, we find long-context LLMs show a **30%~60% performance drop** on LongMemEvalS, and manual evaluations reveal that state-of-the-art commercial systems (such as GPT-4o) only achieve **30%~70% accuracy** in a setting much simpler than LongMemEvalS. Even the most capable long-context LLMs currently would require an effective memory mechanism to manage an ever-growing interaction history.

```
Dataset: longmemeval_s
Model: gpt-4o (Actor) / gemini-2.5-flash (Observer, Reflector)
────────────────────────────────────────────────────────
Accuracy by Question Type:
  knowledge-update          85.9% [                                    ]
(67/78)
  multi-session             79.7% [                                    ]
(106/133)
  single-session-assistant  82.1% [                                    ]
(46/56)
  single-session-preference 73.3% [                                    ]
(22/30)
  single-session-user       98.6% [                                    ]
(69/70)
  temporal-reasoning        85.7% [                                    ]
(114/133)

Overall Accuracy: 84.23%
```

OM is one of the practical answers to "**How do you give an agent long-term memory without paying RAG tax on every turn and without letting tool outputs rot your context?**"

In this article, I'll walk you through its wiring, thresholds, and the failure modes you'll actually see in production.

> *I'm writing a deep-dive ebook on **Agentic SaaS**, the emerging design patterns that are quietly powering the most innovative startups of 2026.*
>
> ***You can grab the first chapter here:** [Agentic SaaS Patterns Winning in 2026](), packed with real-world examples, architectures, and workflows you won't find anywhere else.*

What we're talking about (in one sentence)

Observational Memory (OM) is an agent memory system where **background agents** continuously compress raw message history into a **dense log of observations**, and periodically **garbage collect** those observations so your main agent **remembers** without explicitly querying memory, and without dynamic retrieval.

They built it with 3 things in mind.

1) Tool calls are exploding context windows

Modern agents don't just chat. They **browse**, **screenshot**, **scan repos**, **execute commands**, and **run deep research** in parallel. Those tool results are often massive.

For example, browser agents using Playwright + screenshots, coding agents scanning files + executing commands, deep research agents browsing multiple URLs in parallel.

2) Prompt caching went from "nice" to "mandatory"

Traditional RAG-based "memory" injects dynamic context every turn (embed >> retrieve >> inject). That is great for recall, but it **invalidates your prompt cache constantly**.

In OM, the context is designed to be predictable and stable so you can get full cache hits on most turns, and partial cache hits even when observations update.

3) Bigger context windows did not solve "context rot"

General background knowledge (stable): Even if a model supports a huge context window, performance often degrades as the window fills with irrelevant or noisy tokens ("context rot"). You don't just need "more tokens." You need *better tokens*.

OM is explicitly trying to make small context windows behave like large ones by keeping the context *dense* and *relevant*.

What is Observational Memory, mechanically?

OM splits context into two blocks:

1. **Observations (memory),** a log of what happened, condensed
2. **Raw message history,** the most recent conversation turns that haven't been condensed yet

Messages append to block #2 until you hit a threshold. Then an **Observer agent** compresses those messages into observations, appends them to block #1, and drops the original messages.

When observations grow too large, a **Reflector agent** garbage collects them by combining duplicates, dropping low-value stuff, and reorganizing into a tighter representation.

The core claim is very specific:

- The main agent **never queries** memory.
- The main agent **never writes** memory.
- Background agents do the compression and cleanup.
- The result is a stable prompt prefix that stays cacheable.

Observations look like logs on purpose

OM uses a log-like format instead of structured objects:

- **Formatted text**, not JSON objects or knowledge graphs

- A **three-date model** for better temporal reasoning: observation date, referenced date, relative date
- **Emoji-based prioritization** (effectively log levels):

🔴 important

🟡 maybe important

🟢 informational

Example from coding agent session:

```
Date: 2026-01-15
- 🔴 12:10 User is building a Next.js app with Supabase auth, due in 1 week (meaning January 22nd 2026)
  - 🔴 12:10 App uses server components with client-side hydration
  - 🟡 12:12 User asked about middleware configuration for protected routes
  - 🔴 12:15 User stated the app name is "Acme Dashboard"
```

Copy

You may have notice the usefulness of log levels from.. well logging, simply because:

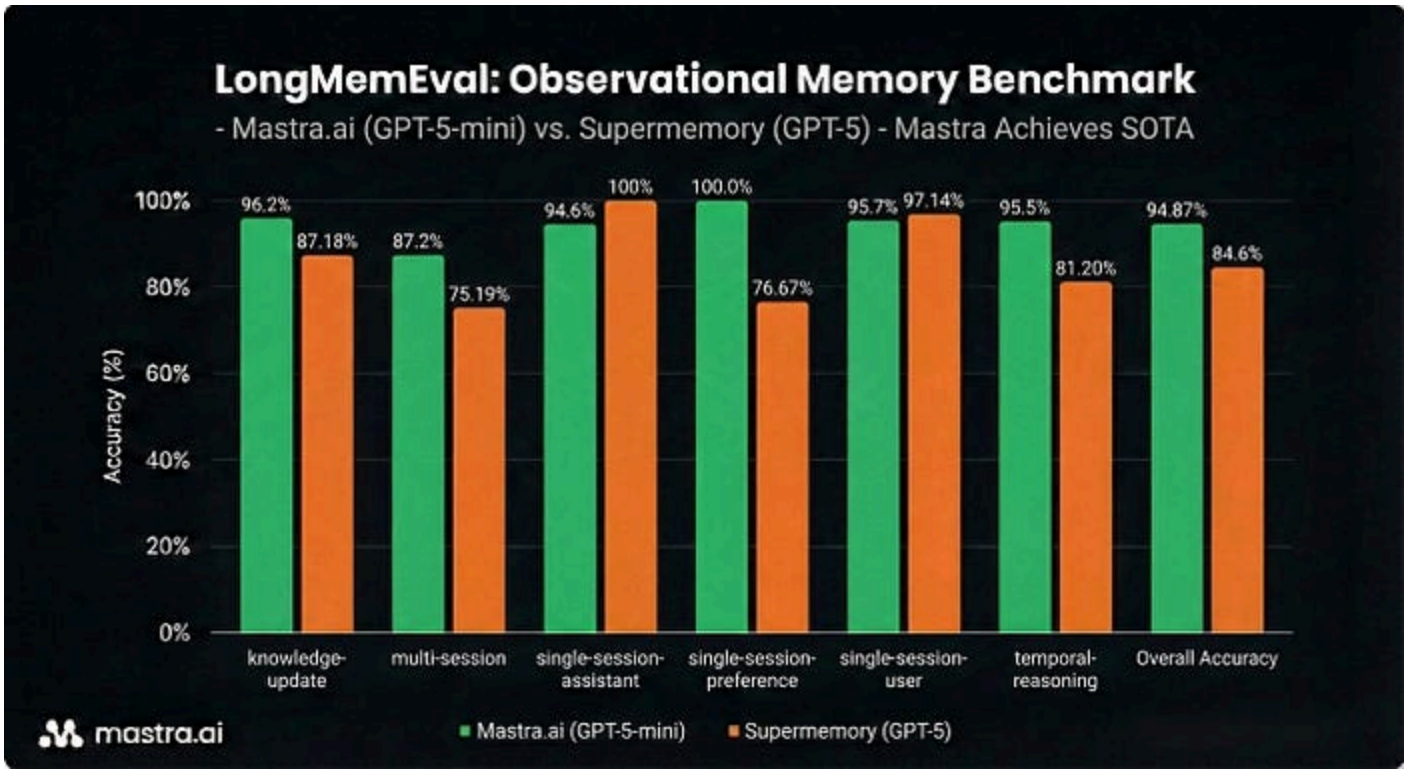
- text is easy to debug (you can read it)
- text is LLM-native (no schema drift)
- text plays well with prompt caching (stable prefix)

The benchmarks

LongMemEval is a memory benchmark with:

- 500 questions
- each question has ~50 sessions attached
- the memory system must enable correct answers across sessions

Mastra OM hits **94.9%** on LongMemEval with **gpt-5-mini** (highest recorded score)



84.2% with **gpt-4o**, beating:

- the **oracle** (82.4%), which only includes the answer sessions
- the previous gpt-4o SOTA from Supermemory (81.6%)
- Mastra RAG (80.05%)
- Zep (71.2%)
- full-context baseline (60.2%)

There are two non-obvious implications here:

1. **Compression can increase accuracy:** OM can be better memory than raw transcripts, because it strips noise.
2. **Stable context is not a performance tax:** Most systems pay for recall by changing the prompt every turn. OM gets high recall while keeping the prompt stable.

Leaderboard

System	Model	Overall
Mastra OM	**gpt-5-mini**	**94.87%**
Mastra OM	**gemini-3-pro-preview**	**93.27%**
Hindsight	gemini-3-pro-preview	91.40%
Mastra OM	**gemini-3-flash-preview**	**89.20%**
Hindsight	GPT-OSS 120B	89.00%
EmergenceMem Internal*	gpt-4o	86.00%
Supermemory	gemini-3-pro-preview	85.20%
Supermemory	gpt-5	84.60%
Mastra OM	**gpt-4o**	**84.23%**
Hindsight	GPT-OSS 20B	83.60%
EmergenceMem Simple	gpt-4o	82.40%

How Observational Memory helps

Let's talk about real failure modes.

My agent forgets, but only after it uses tools

You start with a clean conversation. The agent runs a repo scan or a browser step. Suddenly the context window is full of junk and the model's next answer feels… off.

OM directly targets that by compressing tool-heavy history into a few hundred tokens of observations.

For example Playwright MCP can generate **50,000+ tokens per page snapshot**. OM turns that into a few hundred tokens about what was on the page and what actions were taken.

RAG memory is accurate, but too expensive in production

Semantic recall using RAG scores well, but it's expensive and constantly invalidates prompt cache. OM aims to keep performance high **and** costs sane.

Compaction nukes continuity

OM is designed to make continuity feel "human-like," because it preserves the high-signal narrative instead of bluntly summarizing everything.

Quick start

Let's have a look at few usage examples.

```
import { Memory } from "@mastra/memory";
import { Agent } from "@mastra/core/agent";

export const agent = new Agent({
  name: "my-agent",
  instructions: "You are a helpful assistant.",
  model: "openai/gpt-5-mini",
  memory: new Memory({
    options: {
      observationalMemory: {
        observation: {
          model: "google/gemini-2.5-flash",
        },
        reflection: {
          model: "openai/gpt-4o-mini",
        },
      },
    },
  }),
});
```

That's it.

With `observationalMemory: true`, Mastra defaults the observer/reflector model to **google/gemini-2.5-flash**.

You can customize the observer/reflector model.

```
import { Memory } from "@mastra/memory";
import { Agent } from "@mastra/core/agent";

export const agent = new Agent({
  name: "my-agent",
  instructions: "You are a helpful assistant.",
  model: "openai/gpt-5-mini",
  memory: new Memory({
    options: {
      observationalMemory: {
        model: "openai/gpt-4o-mini",
      },
    },
  }),
});
```

You can also tune token budgets

```
import { Memory } from "@mastra/memory";
import { Agent } from "@mastra/core/agent";

export const agent = new Agent({
  name: "my-agent",
  instructions: "You are a helpful assistant.",
  model: "openai/gpt-5-mini",
  memory: new Memory({
    options: {
      observationalMemory: {
        shareTokenBudget: true,
        observation: {
          messageTokens: 20_000,
          bufferTokens: false, // required when using shareTokenBudget (temporary limitation)
        },
        reflection: {
          observationTokens: 80_000,
        },
      },
    },
  }),
});
```

I suggest to start with defaults unless you have a reason not to. They're conservative and still hit SOTA.

Thread scope is default:

```
const memory = new Memory({
  options: {
    observationalMemory: {
      model: "google/gemini-2.5-flash",
      scope: "thread",
    },
  },
});
```

and there is a **Resource** scope, which is experimental:

```
const memory = new Memory({
  options: {
    observationalMemory: {
      model: "google/gemini-2.5-flash",
      scope: "resource",
    },
  },
});
```

Few things to note:

- Resource scope is marked experimental because it can hurt task adherence across simultaneous threads.
- You may need to tweak your system prompt so one thread doesn't continue work started by another.
- Async buffering is enabled by default but is not supported with `scope: "resource"`

To customize:

```
import { Memory } from "@mastra/memory";
import { Agent } from "@mastra/core/agent";

export const agent = new Agent({
  name: "my-agent",
  instructions: "You are a helpful assistant.",
  model: "openai/gpt-5-mini",
  memory: new Memory({
    options: {
      observationalMemory: {
        model: "google/gemini-2.5-flash",
        observation: {
          messageTokens: 30_000,
```

```
          // Buffer every 5k tokens (runs in background)
          bufferTokens: 5_000,
          // Activate to retain 30% of threshold
          bufferActivation: 0.7,
          // Force synchronous observation at 1.5x threshold
          blockAfter: 1.5,
        },
        reflection: {
          observationTokens: 60_000,
          // Start background reflection at 50% of threshold
          bufferActivation: 0.5,
          // Force synchronous reflection at 1.2x threshold
          blockAfter: 1.2,
        },
      },
    },
  },
});
```

- In resource scope, unobserved messages across all threads are processed together, slow if the user has many existing threads.

You can find [further details about Mastra's memory capabilities here.]

When should you use OM vs RAG vs working memory?

Here's a quick mental model

Need	Best fit	Why
Long-running agent conversations with lots of tools	Observational Memory	Compresses raw history + tool exhaust; stable + cacheable context
Small structured state (names, preferences, goals)	Working memory	Designed for small, structured data; not a growing event log
Retrieve specific past turns based on similarity	Semantic recall (RAG)	Strong recall, but dynamic injection + cache invalidation cost
You're summarizing constantly to survive context limits	Observational Memory	OM manages history automatically via thresholds

If you're currently stuffing **ongoing state that grows over time** into working memory, OM is definitely a better fit.

Shipping OM without getting paged

If you are rolling this into a production agent tomorrow:

1. **Start with thread scope:** Resource scope is tempting, but it's explicitly experimental for task continuity.
2. **Keep defaults for thresholds at first (30k / 40k):** They're conservative and already hit SOTA.
3. **Use async buffering unless you enjoy random pauses:** Disable only if you have a strong reason.
4. **Pick a background model that's consistent and has context headroom** Default is gemini-2.5-flash. Avoid Claude 4.5 for observer/reflector for now.

5. **Treat tool output as toxic waste:** If your agent is a heavy tool user (Playwright screenshots, repo scans), OM is disproportionately valuable.

Thoughts

The frontier is:

- better task adherence across threads
- better safety against prompt injection in long-lived memory logs
- better developer ergonomics for inspecting what the agent **believes**

If dense observations can beat raw transcripts, **what else are we overfeeding models** that we should compress aggressively (tool outputs? intermediate plans? retrieved docs)?

If you're building agents with real tools and real users, I'd love to hear what's currently breaking your memory, and whether the failure mode is recall, cost, or plain old context rot.

Bonus Articles

Local LLMs That Can Replace Claude Code

Small team of engineers can easily burn >$2K/mo on Anthropic's Claude Code (Sonnet/Opus 4.5). As budgets are tight, you...

medium.com

Fully Autonomous Companies: OpenClaw Gateway + Routing + Agents

Fully Autonomous Companies: OpenClaw Gateway + Routing + Agents Whether you think it's hype or not, people are already...

medium.com

ClawRouter: Anthropic charged me $4,660 — How I cut it 70% with smart LLM routing

Last month I opened my credit-card statement and almost threw up. Anthropic charged me $4,660.87, just for AI APIs.

medium.com

Codex 5.3 vs. Opus 4.6: One-shot Examples and Comparison

Codex 5.3 vs. Opus 4.6: One-shot Examples and Comparison Just after 9:45 a.m. Pacific on 5 February 2026, Anthropic…

medium.com

Why Codex Became My Default Over Claude Code (for Now)

If you haven't tried Codex yet, I've got a brief take that might save you a few hours.

medium.com

Clawdbot Lite: 99% Smaller, 4000 Lines, Same Core Power

People are building Polymarket bots, multi-agent product teams, crypto traders, coding agents, business operators with…

medium.com

Vectorless RAG for Agents: PageIndex Is Why Their Demo Works and Yours Needs "Context"

When I'm building a RAG apps, the thing that quietly kills quality (and budgets) is turning a document into a pile of…

medium.com

Fully Local Agentic Coding on localhost: Claude Code, Codex, llama.cpp, Ollama and Docker MR

This is the setup I wish I had the first time I tried to run Claude Code and Codex-style workflows against local…

medium.com

Qwen3-TTS Over ElevenLabs: Open-Source CustomVoice Generation and Voice Cloning via vLLM

Qwen3-TTS Over ElevenLabs: Open-Source CustomVoice Generation and Voice Cloning via vLLM Qwen3-TTS is a suite of…

medium.com
#agent-memory#ai-agent-memory#ai-memory#context-engineering#memory-improvement

Reporting a Problem

Sometimes we have problems displaying some Medium posts.

If you have a problem that some images aren't loading - try using VPN. Probably you have problem with access to Medium CDN (or fucking Cloudflare's bot detection algorithms are blocking you).

Problem Description Describe your problem here...

Submit Cancel